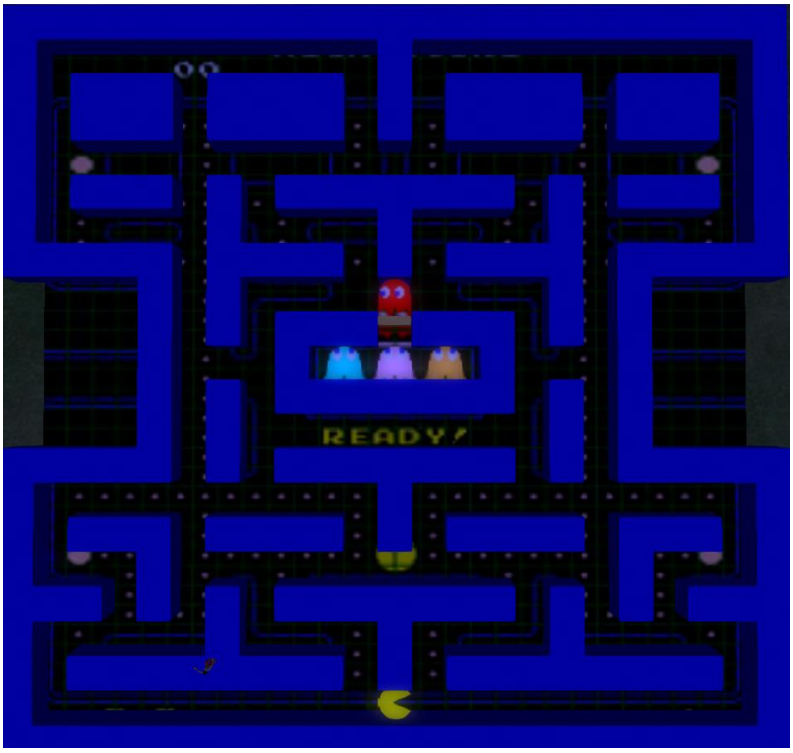
Codeman Ghosts

The Classic Pac-Man AI

In the classic game, ghosts navigate through a **grid-based system** where each ghost constantly targets a specific tile.

- **Pathfinding**: Movement is calculated at every intersection. A ghost looks one tile ahead and decides its next direction by calculating the shortest distance to its current target tile.
- **Directional Restrictions**: Ghosts are prohibited from moving backward, except during a mode change, such as from Scatter to Chase.
- **Grid Snapping**: To ensure smooth navigation through corridors, the script utilizes a `snap` function to align the ghost's position with the `grid_unit` of 2.0 meters.



Ghost Profiles

Each ghost interprets its "target tile" differently depending on its personality and the current game mode.

1. Blinky (The Shadow)

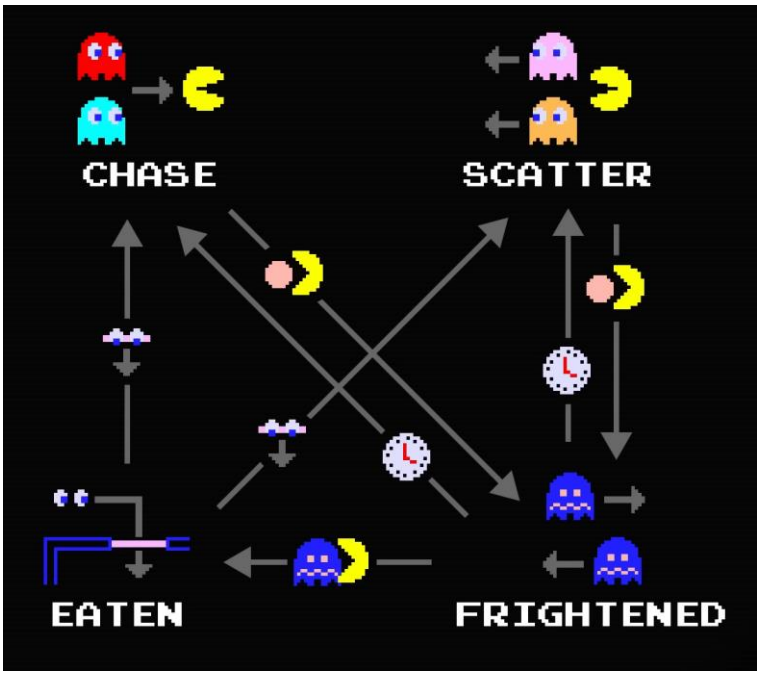
- **Chase Mode**: Blinky is the most direct pursuer; his target tile is Pac-Man's current location.

- **Scatter Mode**: He targets a fixed "Home Corner" located at the top-right of the maze (`blinkyHomeCorner`).

2. Pinky (The Ambusher)
   - **Chase Mode**: Pinky aims to get ahead of the player. Her target tile is four tiles in front of Pac-Man's current direction, attempting to cut the player off.
   - **Scatter Mode**: She retreats to the top-left corner of the maze.
3. Inky (The Bashful)
   - **Chase Mode**: Inky is the most unpredictable of all ghosts. His target depends on both Blinky's position and Pac-Man's position.
   - **Scatter Mode**: He targets the bottom-right corner.
4. Clyde (The Pokey)
   - **Chase Mode**: Clyde has two states based on distance. If he is far away, he behaves like Blinky; however, if he gets too close to Pac-Man, he "cowers" and retreats toward his corner.
   - **Scatter Mode**: He targets the bottom-left corner.

State Switching

The gameplay loop runs inside a high-frequency `timer()` event firing every 0.1 seconds, cycling through 4 main states:



- **Chase**: Standard targeting behavior based on individual ghost personality.
- **Scatter**: Stops chasing Pac-Man and paths toward individual home corners.

Switch between Chase and Scatter is made by GhostController script.

```
default
{
    state_entry()
    {
        llListen(-100, "", NULL_KEY, "");
    }

    listen(integer channel, string name, key id, string msg)
    {
        if (msg == "START_GAME")
        {
            gameActive = TRUE;
            isScatter = TRUE;
            llOwnerSay("Game Logic Started: SCATTER");
            llRegionSay(-200, "SCATTER_MODE");
            llSetTimerEvent(scatterTime);
        }
        else if (msg == "STOP_GAME")
        {
            gameActive = FALSE;
            llSetTimerEvent(0);
            llOwnerSay("Game Logic Stopped.");
        }
    }

    timer()
    {
        if (!gameActive) return;

        if (isScatter)
        {
            llOwnerSay("Switching to: CHASE");
            llRegionSay(-200, "CHASE_MODE");
            isScatter = FALSE;
            llSetTimerEvent(chaseTime);
        }
        else
        {
            llOwnerSay("Switching to: SCATTER");
            llRegionSay(-200, "SCATTER_MODE");
            isScatter = TRUE;
            llSetTimerEvent(scatterTime);
        }
    }
}
```

- **Frightened**: Ignores targeting algorithms, picks a random valid direction at intersections.

  Ghosts become frightened when PowerPellet sends a message through their communication channel.

```
integer isEatean = FALSE;

default
{
    state_entry()
    {
        llListen(-500, "", NULL_KEY, "");
        llCollisionFilter("", NULL_KEY, TRUE);
    }

    listen(integer c, string n, key id, string msg)
    {
        if(msg == "EATEN" && !isEatean)
        {
            llSetAlpha(0.0, ALL_SIDES);
            isEatean = TRUE;

            llCollisionFilter("", NULL_KEY, FALSE);
            llRegionSay(-300, "GHOST_FRIGHT");

            llSetTimerEvent(6.0);
        }
    }

    timer()
    {
        llRegionSay(-300, "GHOST_FRIGHT_STOP");

        llSetAlpha(1.0, ALL_SIDES);
        llCollisionFilter("", NULL_KEY, TRUE);

        isEatean = FALSE;
        llSetTimerEvent(0.0);
    }
}
```

- **Eaten**: Targets `housePosition` directly. Bypasses standard gate collision.

Communication Channels

Each ghost utilizes a `listen` event to manage specific communication streams:

- **Channel -100**: Communication between ghosts and Pac-Man.

```
if (channel == -100)
{
    if (msg == "START_GAME")
    {
        canMove = TRUE;
        llOwnerSay("Ghost Activated!");
    }
    else if (msg == "STOP_GAME")
    {
        ResetGhost();
    }
}
```

- **Channel -200**: Communication between ghosts and the `GhostController`.

```
else if (channel == -200 && !isEaten)
{
    if (msg == "SCATTER_MODE" && isScatterMode == FALSE)
    {
        isScatterMode = TRUE;
        currentDirection = -currentDirection;
        UpdateGhostSprite();
        llOwnerSay("Ghost: Entering Scatter");
    }
    else if (msg == "CHASE_MODE" && isScatterMode == TRUE)
    {
        isScatterMode = FALSE;
        currentDirection = -currentDirection;
        UpdateGhostSprite();
        llOwnerSay("Ghost: Entering Chase");
    }
}
```

- **Channel -300**: Communication between ghosts and PowerPellets.

```
else if (channel == -300)
{
    if (msg == "GHOST_FRIGHT")
    {
        if(isEaten || !isGhostOutside)
            return;

        llOwnerSay("BUU");
        isFrightened = TRUE;
        speed = FRIGHTENED_SPEED;

        currentDirection = llVecNorm(currentDirection) * speed;
        currentDirection = -currentDirection;

        UpdateGhostSprite();
    }
    else if (msg == "GHOST_FRIGHT_STOP")
    {
        if (isEaten || !isGhostOutside)
        {
            isFrightened = FALSE;
            return;
        }

        isFrightened = FALSE;
        speed = NORMAL_SPEED;
        currentDirection = llVecNorm(currentDirection) * speed;

        llSetTextureAnim(FALSE, 0, 0, 0, 0.0, 0.0, 0.0);
        llSetTexture("RedGhost", ALL_SIDES);

        UpdateGhostSprite();
        llOwnerSay("Normal Mode Restored");
    }
}
```

- **Channel -500**: Communication between pacman and PowerPellets.

```
state_entry()
{
    llListen(-500, "", NULL_KEY, "");
    llCollisionFilter("", NULL_KEY, TRUE);
}

listen(integer c, string n, key id, string msg)
{
    if(msg == "EATEN" && !isEatean)
    {
        llSetAlpha(0.0, ALL_SIDES);
        isEatean = TRUE;

        llCollisionFilter("", NULL_KEY, FALSE);
        llRegionSay(-300, "GHOST_FRIGHT");

        llSetTimerEvent(6.0);
    }
}
```

- **Channel 777**: Communication between ghosts and the player's HUD.

```
else if (channel == 777)
{
    if (msg == "SLOW")
    {
        speed = SLOW_SPEED;
        currentDirection = llVecNorm(currentDirection) * speed;
        slowDuration = 5.0;
        llSetColor(<0,0,1>, ALL_SIDES);
    }
}
```

Ghost Reset

Whenever a player fails or exits the maze, a global interrupt is broadcasted over Channel `-100`. Upon receipt, all ghost threads halt runtime loops and invoke `ResetGhost()`, executing the following deterministic safety steps:

1. Stops ghosts (`canMove = FALSE`).
2. Clears state flags (`isEaten = FALSE`, `isFrightened = FALSE`).

3. Resets speed back to `NORMAL_SPEED`.
4. Teleports the ghosts back to `initialPosition` via `llSetRegionPos`.
5. Refresh the default sprite sheet.